

**KSCSC**

Cyber Security Consulting · Offensive Security & Cloud

Penetration Test Report — Web Applications & Cloud

Anonymised work sample · black-box web application assessment

Assessment	Web application & cloud penetration test
Prepared by	KSCSC — Krzysztof Świdrak
Client	Redacted (financial-services organisation)
Approach	Black-box · authenticated & unauthenticated
Classification	Confidential — anonymised work sample

This is a redacted work sample. Client identity, product and host names, tenant names, record identifiers and the specific technology stack have been replaced with neutral placeholders. Proof-of-concept requests are retained to illustrate technique, with hosts and identifiers generalised. Nothing in this document identifies the original engagement.

Contents

1. Executive Summary	3
1.1. Scope of Work	3
1.2. Project Objectives	4
1.3. Assumptions and Threat Model	4
1.4. Timeline	5
1.5. Summary of Findings	5
1.6. Summary of Recommendations	6
1.7. Limitations	6
2. Methodology	7
2.1. Risk Rating	7
3. Detailed Findings	8
3.1. F-001 — Cross-tenant BOLA: any tenant can transfer/exfiltrate another tenant's case	9
3.2. F-002 — CMS BFLA: any authenticated user can delete users and system records via REST and GraphQL	11
3.3. F-003 — Cross-tenant read BOLA: case list returns every tenant's full dossiers	13
3.4. F-004 — Direct browser-to-storage KYC pipeline with no intermediary control layer	15
3.5. F-005 — Paid-order flow: no idempotency and unvalidated lead-routing/branding inputs	17
3.6. F-006 — Unauthenticated read of CMS media and documents collections	18
3.7. F-007 — Missing HTTP security headers across all in-scope applications	19
3.8. F-008 — Stateless JWT session embeds roles/scope and refresh token	20
3.9. F-009 — Onboarding: no server-side authorisation, entitlement, or idempotency	21
3.10. F-010 — Framework disclosure via banner header (CMS)	22
3.11. F-011 — Frontend framework/dependency posture (current and patched)	23
3.12. F-012 — External attack-surface enumeration: unscoped assets and stack profile	24
4. Appendix A — Tools Used	25
5. Appendix B — Glossary	25
6. Penetration Retest Note	25

Penetration Testing Report

1. Executive Summary

This report presents the results of a black-box penetration test conducted against a set of internet-facing web applications and their supporting cloud data plane, operated by a financial-services organisation ("the Client"). The objective was to assess the security posture of the in-scope systems, identify vulnerabilities exploitable by unauthorised or low-privilege authenticated users, and evaluate whether tenant isolation and access-control boundaries are enforced consistently across the application estate.

The assessment identified **one Critical, two High, four Medium, two Low, and three Informational** findings. The most significant risks relate to inconsistent server-side authorisation, including cross-tenant access-control failures that allow authenticated users to read or manipulate data belonging to other tenants. Additional weaknesses were identified in CMS access controls, direct browser-to-storage document handling, input validation, idempotency controls, and baseline HTTP security hardening.

Risk distribution

Critical	High	Medium	Low	Informational
1	2	4	2	3

Overall, the environment demonstrates solid infrastructure and transport-layer hygiene, but application-layer security controls are not enforced consistently across all systems. The Client should prioritise remediation of the cross-tenant authorisation issues and establish a uniform server-side authorisation model, followed by improvements to input validation, entitlement checks, document-handling controls, and security-header baselines across the estate.

Anonymisation notice. This is a redacted work sample. The client identity, product names, hostnames, tenant names, record identifiers, and the specific technology stack have been replaced with neutral placeholders. Proof-of-concept requests are retained to illustrate technique, with hosts and identifiers generalised. Nothing in this document identifies the original engagement.

1.1. Scope of Work

The assessment covered eleven in-scope web applications (referred to here as **App-A** through **App-K**) and their supporting cloud infrastructure, performed under a black-box approach — no prior knowledge of internal architecture, source code, or credentials beyond those issued for role-based testing. Authentication testing used test accounts provisioned for the roles listed below; the identity-provider password-reset page was included as a passive target.

Application	Role (generalised)	Criticality
App-A	Core broker workspace	Highest
App-B	Advisor-handover workflow	Sensitive data
App-C	Case-transfer workflow	Sensitive data
App-D	Case-management / dossier list	Sensitive data
App-E	Headless CMS / marketplace	Highest complexity
App-F	Document / KYC admin	High complexity

Application	Role (generalised)	Criticality
App-G	Staff administration	High complexity
App-H	Partner onboarding	Cost-relevant
App-I	Paid activation / ordering	Cost-relevant
App-J	Learning portal	Standard
App-K	Utility calculator	Lowest risk
IdP	Central SSO / identity provider	Passive testing only

1.1.1. In-Scope Cloud Services

The following cloud components were part of the overall scope. They were not tested in isolation but evaluated in the context of the application landscape: embedded BI (indirect, via host applications), cloud blob storage (access-control review only), the API gateway (access-control review only), the identity provider (passive, OIDC-focused), and the CRM/data platform (context only).

1.2. Project Objectives

The objectives were to evaluate the security posture of eleven internet-facing web applications and their supporting cloud data plane under two contracted threat models (unauthenticated external attacker; authenticated low-privilege attacker), with primary emphasis on **object-level authorisation (IDOR/BOLA)** across the tenant / sales-structure boundary — the engagement's designated high-value class — and on validating that the shared SSO + data-platform architecture enforces tenant isolation consistently. Secondary objectives included transport and header hardening, input-validation posture, session and token design, and an access-control review of the storage / gateway / BI surface. The engagement supported the Client's information-security and financial-sector regulatory obligations.

1.3. Assumptions and Threat Model

Aligned with contemporary regulatory expectations around threat modelling and threat-led testing, the engagement was conducted against three threat scenarios:

1. **Unauthenticated public attacker** — testing without any valid credentials, simulating an external actor probing the public attack surface.
2. **Assumed breach of a least-privilege user (vertical escalation)** — a low-privilege authenticated user attempting to escalate within the same tenant.
3. **Assumed breach of a privileged user (cross-tenant / horizontal escalation)** — an authenticated user of one tenant attempting to access data or functions of another tenant.

General assumptions: an NDA and Rules of Engagement were signed prior to testing; targets were publicly routable staging environments; physical, social-engineering, and denial-of-service attacks were out of scope; cloud platform services were reviewed for access-control posture only. The cross-tenant / horizontal scenario produced the most severe findings (F-001, F-003).

To exercise the cross-tenant scenario, role-based test accounts were provisioned in distinct tenants (banks / sales-structures). Each login authenticates into exactly one tenant. In the authorisation tests a **victim** account owns the target data, and a **challenger** account — deliberately belonging to a different tenant — attempts to read or act on it.

Test login	App context	Owning tenant	Role in testing
user-C1	App-C	Bank-Alpha	Victim — owns F-001 target case

Test login	App context	Owning tenant	Role in testing
user-C2	App-C, App-A	Bank-Beta	Challenger — F-001 cross-tenant attacker
user-D1	App-D	Bank-Delta	Victim — F-003
user-D2	App-D	Bank-Gamma	Challenger — F-003 cross-tenant reader
user-F1	App-F	Structure-2	Victim — holds full data
user-F2	App-F	Structure-3	Challenger
user-B1	App-B, App-A, onboarding	Structure-1	Victim — owns transferable cases
user-B2	App-B	Separate structure	Challenger

Because the owning tenant is fixed per login, any request in which one account reads or operates on data belonging to another account's tenant is, by definition, cross-tenant access. When the server returns or mutates that object without first verifying that the calling account owns the referenced identifier (**caseId**), the missing per-object ownership check is a Broken Object Level Authorisation (BOLA) flaw.

1.4. Timeline

Phase	Start	End
Kick-off / scoping	Day 0	Day 0
Access verification	Day +6	Day +6
Execution	Day +11	Day +18
Report delivery	Day +19	Day +19
Retesting	Later	—

1.5. Summary of Findings

- **F-001 (Critical)** — Cross-tenant broken object-level authorisation in the case-transfer workflow: an authenticated user from one tenant can execute the case-transfer finaliser on another tenant's case and import that tenant's customer financing data. Breaks core tenant isolation in a sensitive-data application.
- **F-002 (High)** — CMS broken function-level authorisation: the admin interface is restricted, but the underlying REST and GraphQL APIs are not. Any authenticated low-privilege user can delete arbitrary CMS user accounts and system records. Impact is destructive rather than privilege-escalating.
- **F-003 (High)** — Cross-tenant read BOLA in the case-management app: the case list is not scoped to the caller's tenant, so any authenticated user can retrieve full customer financing dossiers across tenants in a single request, without enumeration.

Overall posture: the estate demonstrates strong transport, dependency, and infrastructure hygiene. The primary weakness is inconsistent application-layer authorisation and input validation across thin server-action front ends over the shared data platform — a governance and consistency gap rather than an unavoidable platform limitation (several applications implement these controls correctly). None of the identified issues requires sophisticated exploitation; they are reachable by any authenticated low-privilege user.

1.6. Summary of Recommendations

Listed in priority order; a defence-in-depth approach aligned with ISO/IEC 27001:2022, OWASP ASVS, and NIST SP 800-115 is recommended.

- Standardise server-side authorisation across state-changing server actions and data APIs, with deny-by-default access decisions and tenant ownership derived from trusted session state (F-001, F-002, F-003, F-006, F-009).
- Keep refresh-token handling fully server-side and remove long-lived credentials from client-accessible session material (F-008).
- Strengthen input validation and add server-side idempotency / once-only controls for cost-bearing workflows (F-005, F-009).
- Harden the document-handling architecture with an intermediary control layer, ownership checks at token minting, disabled external copy sources, server-side file-type validation, and reduced token permissions and lifetime (F-004).
- Establish a consistent HTTP security-header baseline across all in-scope applications (F-007).
- Resolve supporting hygiene issues: suppress framework banner headers, restrict anonymous CMS media/document access, and confirm server-side package versions (F-006, F-010, F-011).

1.7. Limitations

Penetration testing is time-bounded and resource-bounded, and cannot guarantee that every vulnerability has been discovered. The absence of a finding for a given component must not be read as a guarantee that it is free of other vulnerabilities. To reduce residual risk, a complementary grey-box or white-box test, a dedicated architecture / threat-modelling review, periodic re-testing (at minimum annually and after significant change), and continuous vulnerability and dependency monitoring are recommended.

2. Methodology

The test was performed in three phases: **Planning** (information gathering, live-host detection, reconnaissance, scanning and fingerprinting), **Exploitation** (vulnerability assessment, enumeration, exploitation), and **Reporting** (finding analysis, OWASP Risk Rating, reporting).

2.1. Risk Rating

Risk classification follows the **OWASP Risk Rating Methodology**, deriving overall severity from two dimensions — Likelihood and Impact — each computed from a set of underlying factors (threat-agent skill/motive/opportunity/size; vulnerability ease-of-discovery/ease-of-exploit/awareness/detection; technical impact to confidentiality/integrity/availability/accountability; business impact of financial/reputation/non-compliance/privacy). Each factor is scored 0–9; the group averages give Likelihood and Impact, which map to Low (0–<3), Medium (3–<6), High (6–9). Overall severity is the Likelihood × Impact matrix, with unlikely-and-low-impact combinations treated as informational ("Note").

3. Detailed Findings

Findings are ordered from highest to lowest risk.

ID	Finding	Risk	Affected system
F-001	Cross-tenant BOLA — any tenant can transfer/exfiltrate another tenant's case	CRITICAL	App-C
F-002	CMS BFLA — any authenticated user can delete users/system records (REST + GraphQL)	HIGH	App-E
F-003	Cross-tenant read BOLA — case list returns every tenant's full dossiers	HIGH	App-D
F-004	Direct browser-to-storage KYC pipeline with no intermediary control layer	MEDIUM	App-F → blob storage
F-005	Paid-order flow — no idempotency + unvalidated lead-routing/branding inputs	MEDIUM	App-I
F-006	Unauthenticated read of CMS media and documents collections	MEDIUM	App-E
F-007	Missing HTTP security headers across all in-scope applications	MEDIUM	All 11 apps
F-008	Stateless JWT session embeds roles/scope and refresh token	LOW	All SSO-backed apps
F-009	Onboarding — no server-side authorisation, entitlement, or idempotency	LOW	App-H
F-010	Framework disclosure via banner header (CMS)	INFORMATIONAL	App-E
F-011	Frontend framework/dependency posture (current & patched)	INFORMATIONAL	All 11 apps
F-012	External attack-surface enumeration — unscoped assets & stack profile	INFORMATIONAL	Estate

3.1. F-001 — Cross-tenant BOLA: any tenant can transfer/exfiltrate another tenant's case

Field	Value
Risk Rating	CRITICAL
Calculation	High (Likelihood) × High (Impact) = Critical
Likelihood	6.75 — High
Impact	6.50 — High
Affected system	App-C (case-transfer workflow)
CWE	CWE-639, CWE-863

Description. An authenticated user from one tenant can trigger a business process on data that belongs to a different tenant, breaking expected tenant separation and creating a direct risk of unauthorised access to sensitive customer financing data.

The case-transfer finaliser — `POST /cases/transfer/{caseId}/process` — does not enforce the tenant ownership boundary. An authenticated user belonging to tenant A can execute the full transfer workflow on a case owned by tenant B simply by supplying tenant B's `caseId` in the URL path. The workflow loads the other tenant's customer data, normalises it, imports it into a new case under the attacker's own context, transfers documents, and completes — returning `success:true` end-to-end.

The read path is correctly scoped (`GET /cases` list and `GET /cases/transfer/{caseId}` detail), while the action path (`/process`) is not — a non-owner is not rejected. The same `/process` endpoint on App-B correctly returns 404 to a non-owner, confirming this is an app-specific authorisation gap, not a platform default.

Business Impact. Cross-tenant confidentiality breach (identity, date of birth, address, employment, financial position — regulated PII). Cross-tenant integrity impact: the attacker triggers a full case transfer and imports the victim tenant's customer data into a case under their own context — data exfiltration via the transfer flow. Requires only a low-privilege account in any tenant plus a target `caseId`. If case numbers are guessable, impact broadens to bulk cross-tenant harvesting.

Note. The issue was reported to the Client immediately during the engagement due to its severity for production data, and was potentially remediated in-engagement.

Evidence / Proof of Concept. Two test accounts in different tenants: owner **user-C1** (Bank-Alpha) owns case CASE-AAAA; challenger **user-C2** (Bank-Beta) does not own it and its GET on CASE-AAAA returns no case data.

```
POST /cases/transfer/CASE-AAAA/process (as user-C1, owner) -> HTTP 200, workflow success
POST /cases/transfer/CASE-AAAA/process (as user-C2, other tenant) -> HTTP 200, full success
POST /cases/transfer/CASE-AAAA/process (anonymous) -> HTTP 307 -> /signin
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Cross-tenant POST by user-C2 returning HTTP 200; decoded session token confirms user-C2 identity; case shown transferred

Because user-C1 and user-C2 authenticate into separate tenants, user-C2 driving the `/process` workflow on user-C1's case is cross-tenant access; the server's failure to verify case ownership before acting is the BOLA flaw. The decoded session token confirmed the request executed under

user-C2's identity, and the case was transferred.

Recommendation. Enforce server-side ownership validation on /process and all other state-changing transfer actions so that only the owning tenant can act on a case. Apply fail-closed behaviour for non-owners, consistent with the existing protections on the read path and in App-B. Review all related /cases/transfer/* actions for the same gap and add automated cross-tenant authorisation tests to prevent regression.

References. OWASP ASVS v4.0.3 V4.1.1/V4.1.3/V4.2.1/V4.2.2; OWASP API Security Top 10 2023 API1; OWASP Top 10 2021 A01; WSTG-ATHZ-04; NIST SP 800-53 r5 AC-3/AC-4/AC-6; CWE-639, CWE-863.

3.2. F-002 — CMS BFLA: any authenticated user can delete users and system records via REST and GraphQL

Field	Value
Risk Rating	HIGH
Calculation	High (Likelihood) × Medium (Impact) = High
Likelihood	6.12 — High
Impact	4.50 — Medium
Affected system	App-E (headless CMS)
CWE	CWE-285, CWE-862, CWE-639

Description. Ordinary authenticated users can perform destructive actions in the CMS that should be restricted to administrators. The CMS admin panel is correctly gated (low-privilege users are bounced to /admin/unauthorized, native password login is disabled under SSO-only, first-user takeover is impossible). However, the CMS REST API honours the same SSO session, and the users collection delete access-control function returns true for any authenticated user with no document-level restriction. A lowest-privilege user is therefore authorised to delete arbitrary CMS user accounts (including editor/admin records) via DELETE /api/users. The same over-permissive delete applies to system-jobs and search-results, and was confirmed on GraphQL as well. This is a collection-access-layer flaw, not REST-specific.

The open delete is destructive, not escalatory: create/update on users is blocked and the admin-entry chain is closed by design. Content collections (news, pages, media, documents, etc.) correctly return 403. The flaw is narrowly the users / system-jobs / search-results delete access functions.

Business Impact. Arbitrary CMS user-account deletion by any authenticated user. The users collection mirrors SSO identities; a low-privilege user can delete editor/administrator records, disrupting CMS administration; repeated deletion is a denial-of-service against the user store.

Evidence / Proof of Concept. Authenticated as a lowest-privilege user via SSO; confirmed no admin access (GET /admin -> 307 -> /admin/unauthorized).

```
DELETE /api/users?where[email][equals]=nonexistent-pentest@example.invalid
-> HTTP 200, "0 users deleted" (authorised and executed, removed 0 rows)
DELETE /api/system-jobs -> 200
DELETE /api/search-results -> 200
POST /api/users/login -> 403 (SSO-only)
POST /api/users/first-register -> 403 (already initialised)
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Low-privilege user deleting another account via REST and via GraphQL — both authorised

A worst-case bulk delete using a not_equals filter on a null-UUID would remove all users in one request; it was **not executed** due to its destructive nature. Cross-user deletion of a specific low-privilege account was confirmed via both REST and GraphQL.

Recommendation. Define explicit, role-restricted access controls for the users collection and limit delete/create/update to authorised admin or staff roles. Apply the same to system collections and search-results so destructive operations are consistently protected across REST and GraphQL. Treat the data API as part of the trust boundary and require collection-level server-side access rules for every operation.

References. OWASP ASVS v4.0.3 V4.1.1/V4.1.3/V4.1.5/V4.2.2; OWASP API Security Top 10 2023 API5; OWASP Top 10 2021 A01; WSTG-ATHZ-02; NIST SP 800-53 r5 AC-3/AC-6/AC-6(1); CWE-285, CWE-862, CWE-639.

3.3. F-003 — Cross-tenant read BOLA: case list returns every tenant's full dossiers

Field	Value
Risk Rating	HIGH
Calculation	High (Likelihood) × Medium (Impact) = High
Likelihood	6.88 — High
Impact	5.25 — Medium
Affected system	App-D (case management)
CWE	CWE-639, CWE-863, CWE-200

Description. A user who should only see their own organisation's cases can instead access case data belonging to other tenants. The App-D case list (GET /cases) is not scoped to the caller's tenant. Any authenticated user — regardless of tenant — receives the complete pool of cases across all tenants in the server-rendered response. The inlined payload carries the full customer dossier per case: identity, full address, contact details, employment data, and financial position. No ID guessing or enumeration is required.

Two test accounts in different tenants — **user-D1** (Bank-Delta) and **user-D2** (Bank-Gamma) — receive the identical case set. By contrast, App-C's equivalent read path is correctly tenant-scoped — the gap is App-D-specific.

Business Impact. Bulk cross-tenant breach of full customer financing dossiers. A single low-privilege account in any tenant reads the entire case inventory across all tenants in one request — including each customer's identity, date of birth, full address, email, employment status, and financial position. Provides ready-made target identifiers (caseId) for the action-layer abuse class seen in F-001.

Note. Reported to the Client immediately during the engagement due to severity for production data; potentially remediated in-engagement.

Evidence / Proof of Concept.

```
GET /cases (as user-D1) -> 200, set of 6 cases {CASE-1..CASE-6}
GET /cases (as user-D2) -> 200, byte-identical case-bearing region
GET /cases (anonymous) -> 307 -> /signin
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Two accounts from different tenants receiving the byte-identical full case set

Data classes leaked per case: salutation, first/last name, date of birth, nationality, full address, email, employment status, and financial data (purchase price, amounts, bank details). Both users receive the same six cases despite belonging to different tenants.

Recommendation. Enforce tenant scoping server-side on the case-list query so each user retrieves only cases belonging to their own tenant. Apply the same ownership checks to case detail, document, and all related /cases/* read endpoints. Add automated cross-tenant authorisation tests and re-audit the remaining apps for consistent read scoping.

References. OWASP ASVS v4.0.3 V4.2.1/V4.1.1/V8.1.1; OWASP API Security Top 10 2023 API1; OWASP Top 10 2021 A01; WSTG-ATHZ-04; NIST SP 800-53 r5 AC-3/AC-4/AC-24; CWE-639, CWE-863,

CWE-200.

3.4. F-004 — Direct browser-to-storage KYC pipeline with no intermediary control layer

Field	Value
Risk Rating	MEDIUM
Calculation	Medium (Likelihood) × Medium (Impact) = Medium
Likelihood	5.12 — Medium
Impact	4.38 — Medium
Affected system	App-F → cloud blob storage
CWE	CWE-918, CWE-602, CWE-434

Description. App-F stores contact documents — including identity documents and other KYC PII — in cloud blob storage, moving file bytes directly between the browser and the blob endpoint using a short-lived signed URL. There is no intermediary control layer (no application-tier proxy, no gateway, no WAF) between the client and the storage account on this data plane. Four weaknesses were verified:

- **Facet 1 — Missing object-level authorisation at signed-URL minting (CWE-862/639).** The mint action does not verify the caller owns the supplied contactId. With two accounts in different structures, user-F2 POSTed the mint action for user-F1's contact and received a valid write+read signed URL; a marker PUT to that URL returned HTTP 201. Backstops held (the commit step rejected the cross-tenant attach with 404, and the victim's existing document is unreadable due to UUID-prefixed naming — naming obscurity, not authorisation).
- **Facet 2 — SSRF via copy-from-URL with response read-back (CWE-918).** The upload signed URL permits server-side copy/put-from-URL operations. Out-of-band confirmation was obtained (DNS and HTTPS callbacks originated from cloud provider IP space); the fetched body was stored in the blob and readable via the read URL — full, non-blind SSRF. Boundary: the fetch originates from the storage provider's shared copy engine, not the Client's network, so it cannot reach internal services, private endpoints, or metadata endpoints.
- **Facet 3 — No server-side file-type validation (CWE-434).** File-type enforcement is effectively client-side; text/plain, executables, x-php, text/html, and octet-stream all issued a signed URL. A benign fake-PE PUT returned 201 and read back intact. Arbitrary content (including executables) can reside in the KYC/PII container; SVG stored as .svg is a latent stored-XSS vector (mitigated today by Content-Disposition: attachment).
- **Facet 4 — Use of the storage account access key** is not recommended by the provider; if compromised it grants full account access, with no separation between control-plane and data-plane roles.

Business Impact. The data plane carrying regulated KYC/PII is exposed directly to the internet with no inspection layer, maximising the blast radius of any token leak or future storage-service flaw. A low-privilege user can mint upload tokens for contacts they do not own, use the storage account as an SSRF/open-proxy from the provider's network position, and store arbitrary file types in the container holding customer ID documents. The most damaging escalations (cross-tenant document read/overwrite) are blocked by incidental backstops — hence Medium — but the design relies on those backstops rather than defence-in-depth.

Evidence / Proof of Concept. A standard AV test file (EICAR) was uploaded via the signed URL and confirmed stored; an SSRF copy-source drop was confirmed via an out-of-band collaborator. Signed-URL structure (host, key and signature redacted):

```
https://<storage-account>.<blob-endpoint>/<container>/  
  <uuid>_<uuid>_id-document_<rand>.unknown  
  ?sv=<ver>&se=<expiry>&sr=b&sp=r&sig=<REDACTED>  
  &rscd=attachment%3B%20filename%3D<...>
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Cross-tenant signed-URL mint (201), out-of-band SSRF callback from provider IP space, and EICAR upload stored

Recommendation. Introduce an intermediary control layer for document upload/download so sensitive KYC traffic is no longer handled directly between browser and blob storage. Enforce ownership checks before issuing any signed URL, disable external copy sources, validate file types server-side (MIME + magic-byte), run malware/content scanning, and minimise signed-URL permissions and lifetime. Prefer managed identity over storage account access keys.

References. Cloud Security Benchmark NS-2/PA-7/DP-3/DP-6/LT-4; CIS Cloud Foundations (Storage); OWASP ASVS v4.0.3 V12.1.1/V12.2.1/V12.3.x/V5.2.6/V4.2.1; OWASP API Security Top 10 2023 API1; NIST SP 800-53 r5 SC-7/AC-4/SI-10/AC-3; CWE-918, CWE-602, CWE-434.

3.5. F-005 — Paid-order flow: no idempotency and unvalidated lead-routing/branding inputs

Field	Value
Risk Rating	MEDIUM
Calculation	Medium (Likelihood) × Medium (Impact) = Medium
Likelihood	5.88 — Medium
Impact	3.12 — Medium
Affected system	App-I (paid activation / ordering)
CWE	CWE-20, CWE-79, CWE-840

Description. The App-I purchase/configuration commit (POST /order/summary, a server action) activates a paid monthly subscription (a fixed monthly fee, fulfilled by a third-party lender) and stores the user's lead-capture configuration. The action has **no idempotency / once-only guard** — an already-ordered user can re-submit repeatedly, each returning `success:true`, producing duplicate paid activations. Additionally, no validation is performed on lead-routing and branding fields: `domain1/2/3` accept the `javascript:` scheme, HTML/XSS markup, and non-URLs; `name` accepts HTML; `leadEmail` accepts any address with no format/ownership check. Cross-account access is negative (orders are strictly session-bound with no object-id in the flow).

Business Impact. Duplicate paid activations with no once-only/payment gate — cost abuse. Stored-XSS / open-redirect candidate: unvalidated `domain*/name` fields are the user's lead-capture landing-page config; if rendered unsanitised on the generated public lead page or admin view, `javascript:/HTML` payloads execute against lead submitters or staff. Lead misrouting via arbitrary `leadEmail`. Mass-assignment of disabled UI fields.

Evidence / Proof of Concept. As an authenticated user, captured POST /order/summary and submitted:

```
domain1: "javascript:alert(document.domain)"
domain2: "><img src=x onerror=alert(1)>"
domain3: "https://evil.example.invalid/phish"
leadEmail: "attacker@evil.example.invalid"
-> all accepted, success:true
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Malicious lead-routing values accepted; intruder run of 100 identical submits all returning HTTP 200

Re-submitted 100× in succession via the intruder tool — every response HTTP 200 (no idempotency). Server-side validation confirmed present for consent flag, name (min length), and primaryColor, but absent for URL scheme/format on domains and ownership on leadEmail. The order was confirmed overwritten (downstream CRM status not verifiable within scope).

Recommendation. Strengthen server-side validation for lead-routing and branding inputs (accept only valid HTTP/HTTPS URLs, bind leadEmail to the authorised user, sanitise/encode user-controlled text before storage and rendering). Add server-side idempotency / once-only controls to the paid order flow so repeated submissions cannot create duplicate activations.

References. OWASP ASVS v4.0.3 V5.1.3/V5.1.4/V5.2.x/V11.1.4; OWASP API Security Top 10 2023 API6/API3; OWASP Top 10 2021 A03/A04; NIST SP 800-53 r5 SI-10/SC-5; CWE-20, CWE-79, CWE-840.

3.6. F-006 — Unauthenticated read of CMS media and documents collections

Field	Value
Risk Rating	MEDIUM
Calculation	High (Likelihood) × Low (Impact) = Medium
Likelihood	7.88 — High
Impact	2.38 — Low
Affected system	App-E (headless CMS)
CWE	CWE-639, CWE-200

Description. The CMS exposes two REST collections — media (~672 records) and documents (~172 records) — with read access granted to unauthenticated clients. List endpoints return full record metadata: internal document names, slugs, folder structure, legacy IDs, soft-delete state, and staff user UUIDs (uploadedBy/updatedBy). The collection-level access matrix is also disclosed at /api/access. Sibling collections (/api/pages, /api/role-preferences) correctly return 403, confirming a collection-specific misconfiguration. Image binaries (~487 of 672) download unauthenticated via /api/media/file/{filename} — including internal role-rights matrices. A pagination-cap bypass (limit=0) returns the entire collection in one request. A second vector was confirmed via unauthenticated GraphQL.

Business Impact. An unauthenticated internet attacker can enumerate the complete CMS document/media inventory and data model, harvest staff user UUIDs for follow-on BOLA testing, map content structure and sequential legacy ID ranges, and download internal authorisation matrices — a direct aid to F-001/F-003. No customer PII observed in sampled records.

Evidence / Proof of Concept.

```
GET /api/documents?limit=0 (unauthenticated) -> 200, totalDocs:172, incl. staff UUIDs
GET /api/media?limit=0 (unauthenticated) -> 200, totalDocs:672
GET /api/access (unauthenticated) -> 200, full anonymous permission matrix
GET /api/media/file/<redacted>.png -> 200
query{ Documents{ totalDocs docs{ id name } } } (GraphQL, unauth) -> 200
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Unauthenticated media/documents listing and the anonymous permission matrix at /api/access

Recommendation. Restrict read access to media and documents so only authenticated users with appropriate role/tenant scope can retrieve records. If some media must remain public, expose only a published subset through a field-limited endpoint that omits operational metadata. Restrict anonymous access to /api/access, exclude soft-deleted content, enforce a hard pagination limit, and apply the fix at the collection access-control layer so REST and GraphQL are both covered.

References. OWASP ASVS v4.0.3 V4.1.1/V4.2.1/V8.1.1/V13.1.1/V13.2.x/V14.3.x; OWASP API Security Top 10 2023 API1/API3; OWASP Top 10 2021 A01; WSTG-ATHZ-04; NIST SP 800-53 r5 AC-3/AC-24; CWE-639, CWE-200.

3.7. F-007 — Missing HTTP security headers across all in-scope applications

Field	Value
Risk Rating	MEDIUM
Calculation	High (Likelihood) × Low (Impact) = Medium
Likelihood	7.38 — High
Impact	2.62 — Low
Affected system	All 11 in-scope applications
CWE	CWE-1021, CWE-693

Description. Application responses (both unauthenticated redirects and authenticated rendered pages) served by every in-scope app omit the standard HTTP security headers. There is no Content-Security-Policy, no X-Frame-Options/frame-ancestors, no X-Content-Type-Options, no Referrer-Policy, no Permissions-Policy, and no Strict-Transport-Security on the app origins. The only headers returned are Content-Type, Cache-Control, and a framework banner. The IdP login page is correctly hardened; the application origins are not. CORS was tested and is **not** misconfigured (no Access-Control-Allow-Origin for any probe).

Business Impact. Clickjacking of authenticated pages including admin/state-changing functions; no CSP = no defence-in-depth against XSS (compounds F-008); referrer leakage of sensitive identifiers (e.g. the caseId exploited in F-001) to third parties; MIME sniffing without nosniff (relevant to F-006 media); missing HSTS leaves a TLS-stripping window on first contact.

Evidence / Proof of Concept.

```
GET / (authenticated) -> HTTP 200 text/html
Response headers: content-type, cache-control, x-powered-by (only)
Absent: CSP, XFO/frame-ancestors, X-Content-Type-Options, Referrer-Policy,
Permissions-Policy, HSTS
Same result across all 11 apps.
CORS probe with arbitrary Origin -> no Access-Control-Allow-Origin (negative)
```

[ **SCREENSHOT REDACTED** — PoC evidence held in the original]

Authenticated response headers showing only content-type / cache-control / banner — no security headers

Recommendation. Establish a standard security-header baseline for all applications at the shared edge and, where appropriate, in the application framework — at minimum CSP, X-Frame-Options / frame-ancestors, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, and HSTS. Validate the policy centrally and apply it consistently.

References. OWASP ASVS v4.0.3 V14.4.x/V3.4.1-3.4.3/V9.1.1; OWASP Top 10 2021 A05; OWASP Secure Headers Project; WSTG-CONF-07/WSTG-CLNT-09; NIST SP 800-53 r5 SC-8/CM-6; CWE-1021, CWE-693.

3.8. F-008 — Stateless JWT session embeds roles/scope and refresh token

Field	Value
Risk Rating	LOW
Calculation	Medium (Likelihood) × Low (Impact) = Low
Likelihood	4.88 — Medium
Impact	2.62 — Low
Affected system	All SSO-backed apps; App-E CMS (DOM exposure)
CWE	CWE-522, CWE-539

Description. The application session is a stateless, self-contained JWT (HS256) stored in a cookie. Its payload carries the user's identity, authorisation data (roles, scope) and the IdP refresh token (a long-lived credential). The cookie is correctly flagged Secure; HttpOnly; SameSite, and the signing secret resisted a 14.3M-word offline dictionary attack. The concern is the design: a bearer token that both encodes authorisation claims client-side and carries a refresh token concentrates risk. On App-E, the session including the full refresh token is serialised into the client-hydration stream of every authenticated page — readable by any JavaScript on the origin (the HttpOnly flag blocks `document.cookie`, not JS reads of inlined page data). However, the OIDC clients are confidential: redeeming the refresh token requires the client secret (server-side only), so a stolen refresh token cannot on its own be exchanged for access tokens. Impact is session-hijack (while the victim's session lives), not durable IdP takeover.

Business Impact. On its own: Low (cookie is HttpOnly/Secure/SameSite and the signing secret is strong). Compounded: any XSS or token-leak on these apps escalates to session hijack, because the session JSON incl. refresh token is readable from the hydration DOM on the CMS. Does not yield secret-independent token minting or persistent IdP access (confidential OIDC clients). Stateless design makes server-side revocation harder.

Evidence / Proof of Concept. Authenticated to an SSO-backed app; read the session cookie; base64url-decoded the JWT payload -> observed `user.roles`, `user.businesses`, and `refreshToken`. On the CMS the refresh token is present in the page's hydration data, so any JS on the origin can read it (cookie attributes are not effective here; note the absence of CSP).

[ SCREENSHOT REDACTED — PoC evidence held in the original]

Decoded session JWT with refresh token; the same token read from the page hydration data via the JS console

Recommendation. Keep refresh tokens fully server-side and remove them from the client session cookie and any client-readable DOM/hydration data. Prefer a backend-for-frontend or opaque server-side session model so long-lived credentials are never exposed to the browser. Continue to enforce tenant/business scoping from trusted server-side state and maintain strong key-management for session signing.

References. OWASP ASVS v4.0.3 V3.5.2/V3.5.3/V8.1.1/V8.3.x; OWASP Top 10 2021 A02/A07; Cloud Security Benchmark IM-6; NIST SP 800-53 r5 IA-5/SC-28; RFC 9700 (OAuth refresh-token BCP); CWE-522, CWE-539.

3.9. F-009 — Onboarding: no server-side authorisation, entitlement, or idempotency

Field	Value
Risk Rating	LOW
Calculation	Medium (Likelihood) × Low (Impact) = Low
Likelihood	5.62 — Medium
Impact	2.75 — Low
Affected system	App-H (onboarding)
CWE	CWE-639, CWE-915, CWE-602

Description. The cost-relevant onboarding submission (/onboarding/summary) trusts a fully client-supplied payload and performs only structural (schema) validation + consent checks. No server-side authorisation, entitlement, identity-binding, or idempotency at the action layer. An authenticated user can submit a roles map assigning roles to identifiers outside their own structure, self-enable paid lead features, supply arbitrary company/identity fields, and re-submit repeatedly — each returning success:true. CRM verification confirmed the success:true submissions did **not** persist to the master record — onboarding is intake/approval-gated downstream.

Business Impact. Potential for duplicate cost/activation events via repeated submission (no once-only guard). Roles map accepts foreign identifiers (cross-structure role-assignment attempt). Self-activation of paid features without entitlement check. Mitigated by the downstream approval gate — master record not altered. Defence-in-depth and duplicate-intake cost risk.

Evidence / Proof of Concept. As an authenticated user, captured POST /onboarding/summary; injected a foreign identifier as lead-manager -> success:true (no ownership rejection); repeated several times -> success:true each time.

[ SCREENSHOT REDACTED — PoC evidence held in the original]

Onboarding submission with a foreign identifier accepted; repeated submits each returning success

Recommendation. Enforce server-side authorisation on the roles map so only identifiers belonging to the caller's own structure are accepted. Add server-side entitlement checks for paid features, bind company/identity data to trusted session or CRM records rather than client-supplied values, and add idempotency / once-only controls.

References. OWASP ASVS v4.0.3 V4.2.1/V5.1.1/V5.1.2/V11.1.4; OWASP API Security Top 10 2023 API1/API3/API6; OWASP Top 10 2021 A01/A04; NIST SP 800-53 r5 AC-3/SI-10; CWE-639, CWE-915, CWE-602.

3.10. F-010 — Framework disclosure via banner header (CMS)

Field	Value
Risk Rating	INFORMATIONAL
Calculation	High (Likelihood) × Low (Impact) = Informational
Likelihood	8.00 — High
Impact	1.25 — Low
Affected system	App-E (headless CMS)
CWE	CWE-200

Description. The CMS returns a banner header on all responses, disclosing the underlying web framework and the headless CMS product. The other ten in-scope applications suppress this header. Disclosure narrows an attacker's research to framework- and product-specific vulnerabilities and admin surfaces.

Business Impact. Information disclosure that assists reconnaissance and vulnerability targeting; no direct compromise. Best read together with F-006 (the REST/GraphQL surface it points an attacker toward).

Evidence / Proof of Concept. GET / returns a framework/product banner header, observed on /, /admin, /api/*, and error responses.

[ SCREENSHOT REDACTED — PoC evidence held in the original]

Response header disclosing the web framework and headless-CMS product

Recommendation. Disable framework-identifying response headers in the CMS so it matches the other applications. Verify at both origin and edge that no additional identifying or version headers are exposed.

References. OWASP ASVS v4.0.3 V14.4.4/V14.3.3/V14.2.1; OWASP Top 10 2021 A05; OWASP Secure Headers Project; WSTG-INFO-08; NIST SP 800-53 r5 CM-6/SC-30; CWE-200.

3.11. F-011 — Frontend framework/dependency posture (current and patched)

Field	Value
Risk Rating	Informational (positive posture note)
Likelihood	4.00 — Medium
Impact	1.25 — Low
Affected system	All 11 in-scope applications
CWE	CWE-1104, CWE-1035

Description. Black-box assessment of the client-side framework stack found the applications running the fully-patched framework releases addressing the coordinated security release and server-component vulnerability chain disclosed earlier in the year. No vulnerable framework version was observed, and both relevant unauthenticated auth-bypass exploits were tested and do not work. Eight of eleven apps were string-confirmed; three are behaviourally identical but not string-confirmed (bundler output). Source maps are not exposed.

Business Impact. Positive posture — no vulnerable framework version observed. This item documents the assessment and one residual confirmation item.

Recommendation. Maintain the current patch cadence and dependency-monitoring discipline. As a supporting assurance measure, validate server-side package versions through software inventory or SBOM where direct black-box confirmation is not possible.

References. OWASP ASVS v4.0.3 V14.2.x; OWASP Top 10 2021 A06; Cloud Security Benchmark PV-3/PV-5/PV-6; NIST SP 800-53 r5 SI-2/RA-5/SR-3/SR-4; CWE-1104, CWE-1035.

3.12. F-012 — External attack-surface enumeration: unscoped assets and stack profile

Field	Value
Risk Rating	INFORMATIONAL
Calculation	High (Likelihood) × Low (Impact) = Informational
Likelihood	7.50 — High
Impact	1.62 — Low
Affected system	Estate (public sources)
CWE	CWE-200, CWE-668

Description. A combination of passive reconnaissance (public DNS, Certificate-Transparency logs, search engines/archives, WHOIS, cloud global-namespace existence checks — no asset contacted) and active fingerprinting (response headers, JS bundle inspection, OIDC discovery — all within normal browser behaviour on in-scope hosts) produced four classes of result:

1. **Discovered-but-unscoped applications** sharing the agreed ingress (four additional staging hostnames, incl. a likely reporting/BI surface). Not in engagement scope.
2. **Cloud resource candidates** (DNS-existence only), including storage accounts used directly by in-scope apps and confirmed during the engagement.
3. **Identity-surface leakage** — the IdP realm exposes its OIDC `client_id` set publicly; one client lacks PKCE.
4. **Technology-stack fingerprinting** — the full platform profile is attributable from public signals within roughly one day of passive recon.

Business Impact. No direct compromise (passive). The value is attack-surface visibility: an external actor can, from public sources alone, map the application estate, the cloud footprint, and the identity surface — and identify unscoped apps on the same ingress/realm that may carry the same systemic flaws found in scope. The reporting/BI host is the most material: if it is the reporting surface, it is the likeliest place for sensitive cross-tenant data — and it was not tested.

Recommendation. Maintain a current external asset inventory and review DNS, certificate-transparency, and public cloud exposure regularly so newly visible services are brought into scope or formally monitored. For internet-facing cloud resources, avoid predictable naming patterns derived from application names, environments, or business functions.

References. OWASP ASVS v4.0.3 V1.1.2/V14.2.x; OWASP WSTG WSTG-INFO-01/02/04; MITRE ATT&CK TA0043/TA0007/TA0001; Cloud Security Benchmark AM-1/AM-2/NS-2/GS-2; CIS Cloud Foundations Benchmark; NIST SP 800-53 r5 CM-8/RA-5/SC-7; CWE-200, CWE-668.

4. Appendix A — Tools Used

Topic	Tooling (generalised)
DNS / passive recon	dig/delv, CT-log search, subdomain + certificate enumeration, WHOIS/RDAP
Cloud attribution	Service-tag and global-namespace DNS-existence checks (passive)
URL / content	HTTP probing, content enumeration, archived-URL collection
Headers / TLS	Header + TLS posture assessment (nmap NSE, testssl-class tools, curl)
Web app testing	Intercepting proxy — repeater, intruder, out-of-band collaborator
Injection	Automated SQLi testing (negative on the data path)
Auth / crypto	Offline session-secret dictionary attack, OIDC-flow and JWT decode/forged tests
Analysis	Local LLM models for response/code analysis and payload/wordlist generation

5. Appendix B — Glossary

Term	Definition
BOLA	Broken Object Level Authorization — the server fails to verify that the authenticated user owns the requested resource (cf. IDOR).
BFLA	Broken Function Level Authorization — access-control checks on privileged or admin functions are missing or bypassable.
IDOR	Insecure Direct Object Reference — an object is addressed by identifier with no ownership check; the practical symptom of a BOLA flaw.
OWASP Risk Rating	The qualitative Likelihood × Impact framework used as the primary severity model in this report.
CWE	Common Weakness Enumeration — a public catalogue of general weakness types referenced throughout the findings.

6. Penetration Retest Note

Dedicated retesting credentials were provided to validate the three most significant findings. **All retested findings were confirmed as fixed.**

ID	Finding	Risk	Status
F-001	Cross-tenant BOLA (case-transfer /process)	CRITICAL	Fixed, Verified
F-002	CMS BFLA (delete via REST + GraphQL)	HIGH	Fixed, Verified
F-003	Cross-tenant read BOLA (case list)	HIGH	Fixed, Verified

F-001 — Logged in as owner user-C1, transferable cases were visible; replacing the session cookie with challenger user-C2 now returns **404 Not Found** for those cases — the non-owner has no access.

F-002 — Logged in as a low-privilege user, DELETE on /api/users, /api/system-jobs, and /api/search-results is now rejected. (Note raised: GET /api/search-results still returns some data)

— worth confirming whether it should be readable.)

F-003 — Intercepting a GET as user-D1 and replacing the cookie for user-D2 now returns a response containing **no** cases belonging to user-D1; comparison confirmed only a "digest" was returned to the second user rather than the full data chunk. (Note raised: a 500 error occurs on a POST for a non-existent case — worth validating the processing logic; and consider returning 404/403 rather than an empty 200.)